

mobiTrace

Arbitrage criticité des données

Niveau de sécurisation applicatif

Référence	TRACE-SEC-DATA-2026
Version	1.0
Date	08/08/2026
Auteur	Équipe technique mobiTrace / DGFIP

Résumé exécutif

Ce document établit l'arbitrage de criticité des données traitées par mobiTrace, application souveraine de gestion d'inventaire mobilier développée par et pour la DGFIP. L'analyse suit la grille CIA (Confidentialité / Intégrité / Disponibilité) et les niveaux de classification ANSSI (Public / Interne / Sensible / Critique).

Résultat : mobiTrace traite principalement des données de niveau Interne à Sensible, sans secret d'État ni donnée financière. Deux actifs ressortent au niveau Critique : le secret JWT et les dumps de sauvegarde. Ce sont les priorités de sécurisation immédiates. Trois points de risque structurels sont identifiés avec leurs remédiations.

Avertissement — Ce document contient une évaluation de la surface de risque de l'infrastructure. Ne pas diffuser hors des équipes habilitées.

1. Cadre d'analyse

1.1 Grille CIA

Chaque catégorie de données est évaluée selon trois axes indépendants, sur trois niveaux :

Axe	Niveaux	Définition
Confidentialité (C)	Faible / Élevée /	Impact d'une divulgation non autorisée
Intégrité (I)	Faible / Élevée /	Impact d'une altération ou falsification
Disponibilité (D)	Faible / Élevée /	Impact d'une indisponibilité ou perte

1.2 Niveaux de classification (ANSSI)

Niveau	CIA max	Exemples de données
Public	C=Faible / I=Faible	Catalogue gabarits, référentiels publics
Interne	C=Faible / I=Élevée	Données mobilier, référentiels organisationnels
Sensible	C=Élevée / I=Critique	Comptes utilisateurs, journal d'audit
Critique	C=Critique / I=Critique	Secret JWT, dumps de sauvegarde

1.3 Périmètre de l'application

mobiTrace est une application souveraine déployée en intranet DGFIP, sans exposition internet. Les utilisateurs sont exclusivement des agents publics authentifiés. Le code source est public (CC BY-NC-SA) mais les données de production restent strictement internes.

- Pas de données financières (aucun lien avec les systèmes comptables)
- Pas de secret de défense nationale
- Données personnelles au sens RGPD : emails et noms des agents uniquement
- Données patrimoniales : inventaire de mobilier et équipements administratifs
- Données opérationnelles : affectations, statuts, journal d'actions

L'architecture minimaliste (pas de framework applicatif côté serveur, logique métier en base PostgreSQL) réduit significativement la surface d'attaque par rapport à une application standard en Node.js ou Python Flask.

2. Inventaire et cotation des données

2.1 Secret JWT (auth.secrets)

Critère	Niveau	Justification	Niveau global
Confidentialité	Critique	Fuite = forge illimitée de tokens, y compris rôle a	CRITIQUE
Intégrité	Critique	Modification = toutes les sessions actives cassées	
Disponibilité	Élevée	Indisponibilité = plus aucun accès à l'application	

Risque maximal : un attaquant en possession de ce secret peut se connecter en administrateur sans credentials, sans laisser de trace dans les logs d'authentification. Actuellement stocké dans la même base PostgreSQL que les données applicatives.

2.2 Dumps de sauvegarde NFS

Critère	Niveau	Justification	Niveau global
Confidentialité	Critique	Contient l'intégralité de la BDD : hashes mdp, sec	CRITIQUE
Intégrité	Élevée	Restauration sur dump corrompu = perte de donn	
Disponibilité	Élevée	Seul filet de sécurité post-incident	

Le dump PostgreSQL est une copie exacte de toutes les données sensibles. Il circule actuellement en clair sur le réseau NFS (réseau interne non chiffré). La compromission du serveur NFS équivaut à la compromission complète de l'application.

(suite — Inventaire et cotation des données)

2.3 Table utilisateurs (comptes agents)

Critère	Niveau	Justification	Niveau global
Confidentialité	Élevée	Emails + noms d'agents = données personnelles	SENSIBLE
Intégrité	Critique	Modification = escalade de privilèges, création de	
Disponibilité	Élevée	Perte = blocage total des accès applicatifs	

Données personnelles au sens RGPD (emails nominatifs agents DGFIP). Le REVOKE explicite sur cette table est une bonne mesure en place. L'administrateur applicatif conserve cependant un accès complet.

2.4 Journal d'audit (audit_logs)

Critère	Niveau	Justification	Niveau global
Confidentialité	Moyenne	Révèle l'activité opérationnelle des agents (qui to	SENSIBLE
Intégrité	Critique	Table modifiable par l'admin : valeur probatoire n	
Disponibilité	Faible	Consultation a posteriori, non bloquant en exploit	

Paradoxe actuel : l'audit log est stocké dans la même base que les données auditées, accessible à l'administrateur applicatif qui peut donc couvrir ses propres actions. La valeur probatoire est nulle en cas de litige sans externalisation du journal.

2.5 Données mobilier (mobiliers)

Critère	Niveau	Justification	Niveau global
Confidentialité	Faible	Inventaire de mobilier, aucune donnée personnelle	INTERNE
Intégrité	Élevée	Falsification = inventaire non fiable, risque de pe	
Disponibilité	Moyenne	Indisponibilité quelques heures acceptable, activi	

Données de gestion administrative courante. Sensibilité limitée à l'intégrité opérationnelle. Le RBAC PostgreSQL (agent/administrateur/lecteur) et les triggers d'audit constituent une protection adéquate pour ce niveau.

2.6 Référentiels organisationnels (structures, lieux)

Critère	Niveau	Justification	Niveau global
Confidentialité	Faible	Codes SAGES et libellés de structures DGFIP, sem	INTERNE
Intégrité	Élevée	Erreur = mauvaise affectation géographique des	
Disponibilité	Faible	Données quasi-statiques	

Référentiels d'organisation administrative. La charte de nommage stricte et l'import CSV contrôlé à l'installation réduisent le risque d'altération accidentelle.

2.7 Catalogue gabarits (gabarits)

Critère	Niveau	Justification	Niveau global
Confidentialité	Faible	Références catalogue mobilier standard, photos b	INTERNE
Intégrité	Moyenne	Modification = mauvaise identification du matéri	
Disponibilité	Faible	Donnée de référence, lecture prédominante	

Catalogue de gabarits d'équipements. Données non sensibles, protégées par le RBAC (seul l'administrateur peut créer ou modifier les gabarits).

3. Tableau de bord de synthèse

Catégorie de données	Conf.	Intég.	Dispo.	Niveau	Mesures clés en place
Secret JWT	C3	C3	C2	CRITIQUE	auth.secrets, SECURITY DEFINER
Dumps NFS	C3	C2	C2	CRITIQUE	Cron 2x/jour, sonde email
Comptes utilisateurs	C2	C3	C2	SENSIBLE	Bcrypt, REVOKE, RBAC
Journal d'audit	C1	C3	C1	SENSIBLE	Trigger automatique
Données mobilier	C1	C2	C2	INTERNE	RBAC agent/lecteur, UUID
Référentiels org.	C1	C2	C1	INTERNE	Import CSV contrôlé
Catalogue gabarits	C1	C1	C1	INTERNE	RBAC administrateur

Répartition par niveau



Conclusion : mobiTrace est une application à sensibilité globale INTERNE à SENSIBLE. Aucune donnée de niveau Secret défense ni financière. Les deux actifs Critique (secret JWT et dumps) sont des vecteurs d'amplification de risque qui, compromis, élèvent de fait le niveau de l'ensemble de l'application.

4. Points de risque et recommandations

4.1 Priorité 1 — Critique (risque immédiat)

Dumps NFS en clair

Le dump PostgreSQL (contenant le secret JWT et les hashes de mots de passe) transite sur le réseau en clair via NFS. Sur un réseau d'administration partagé, une écoute passive suffit à capturer l'intégralité de la base.

- Remédiation : chiffrer les dumps avant transfert avec une clé symétrique AES-256

Exemple : Commande dans `trace_backup.sh` :

```
# Dans trace_backup.sh -- chiffrer avant copie NFS
openssl enc -aes-256-cbc -pbkdf2 -iter 600000 \
  -in /tmp/trace_backup_${DATE}.sql.gz \
  -out /mnt/savetrace/trace_backup_${DATE}.sql.gz.enc \
  -pass file:/etc/trace-backup.key # clé root:root 600
```

Journal d'audit falsifiable par l'administrateur

La table `audit_logs` réside dans la même base que les données auditées. L'administrateur applicatif peut modifier ou supprimer des entrées, annulant toute valeur probatoire en cas de litige ou d'incident de sécurité.

- Remédiation court terme : activer `pgaudit` (extension PostgreSQL) vers `syslog` distant en mode `push-only` (l'administrateur applicatif ne peut pas y accéder)
- Remédiation long terme : externaliser vers un SIEM ou `syslog` centralisé sous contrôle de l'équipe sécurité DGFIP

Secret JWT colocalisé avec les données

Le secret JWT est stocké dans `auth.secrets` dans la même instance PostgreSQL que les données. Une injection SQL sur une vue ou une fonction `SECURITY DEFINER` pourrait l'exposer.

- Remédiation : stocker le secret dans une variable d'environnement `systemd` (directive `Environment=` dans `trace-api.service`), hors de la base de données

4.2 Priorité 2 — Sensible

Certificat SSL auto-signé

Le certificat auto-signé généré à l'installation est identifié dans le README comme devant être remplacé. Les navigateurs affichent une alerte SSL, et le certificat ne peut pas être révoqué via une CA.

- Remédiation : demander un certificat auprès de l'IGC/A (IGC de l'administration)
- Alternative interne : déployer une CA interne DGFIP et distribuer le certificat racine

Absence de second facteur pour les administrateurs

Un compte administrateur compromis (phishing, mot de passe faible ou réutilisé) donne un accès complet à l'ensemble de l'application, y compris la table `utilisateurs` et les fonctions `SECURITY DEFINER`.

- Remédiation : implémenter TOTP (Google Authenticator / FreeOTP) pour le rôle administrateur
- La fonction de login PostgreSQL peut vérifier un code TOTP via `pgcrypto` HMAC-SHA1

Révocation de JWT impossible

L'architecture `stateless` JWT ne permet pas d'invalidier un token avant son expiration (8 heures). Si un agent quitte le service ou si un compte est compromis, le token reste valide jusqu'à expiration naturelle.

- Remédiation : réduire la durée des tokens de 8h à 2h
- Remédiation avancée : tenir une liste noire en mémoire (table PostgreSQL légère)

4.3 Priorité 3 — Amélioration

RGPD — Données personnelles des agents

Les emails et noms complets des agents constituent des données personnelles au sens du RGPD. L'application ne dispose pas d'un mécanisme de suppression des comptes pour les agents quittant le service.

- Vérifier que le traitement est couvert par le registre des traitements DGFIP
- Implémenter une procédure de désactivation/anonymisation des comptes agents partants
- Durée de conservation des audit_logs à définir et configurer (purge automatique)

Rotation du secret JWT

Aucun mécanisme ni procédure de rotation planifiée du secret JWT n'est documenté. Un secret non rotatif depuis l'installation est exposé en cas de compromission silencieuse.

- Documenter la procédure de rotation (UPDATE auth.secrets + redémarrage PostgREST)
- Planifier une rotation annuelle au minimum

5. Plan d'action

Prio.	Action	Effort	Impact	Responsable
P1	Chiffrer les dumps NFS (openssl enc	0.5 j	Critique	Équipe technique
P1	Externaliser le secret JWT hors BDD	0.5 j	Critique	Équipe technique
P1	Activer pgaudit vers syslog distant	1 j	Critique	Équipe technique
P2	Remplacer le certificat SSL auto-sign	0.5 j	Élevé	Équipe technique
P2	TOTP pour le rôle administrateur	2 j	Élevé	Équipe technique
P2	Réduire durée JWT de 8h à 2h	0.5 j	Moyen	Équipe technique
P3	Procédure RGPD : désactivation com	1 j	Moyen	Équipe technique
P3	Procédure de rotation annuelle du se	0.5 j	Faible	Équipe technique

Visualisation temporelle du plan d'action



Mesures de sécurité déjà en place

Les éléments suivants constituent les points forts de la sécurité actuelle :

- Hachage Bcrypt (pgcrypto, sel aléatoire) pour les mots de passe utilisateurs
- JWT HttpOnly / Secure / SameSite=Strict — protection XSS et CSRF
- Fail2Ban sur POST /api/rpc/login : ban 1h après 5 tentatives (10 min)
- Rate limiting Nginx : zone login_limit 3 req/s, réponse 429
- RBAC PostgreSQL strict : rôles agent / administrateur / lecteur
- REVOKE explicite sur table utilisateurs pour les rôles non-admin
- SECURITY DEFINER sur les fonctions sensibles (login, creer_utilisateur)
- Journal d'audit intégré avec triggers automatiques
- Sauvegarde automatique 2x/jour avec sonde de surveillance par email
- Génération des secrets (JWT, mdp BDD) à l'installation via /dev/urandom
- Architecture sans framework applicatif côté serveur (surface d'attaque minimale)

Document généré automatiquement par `generate_arbitrage_criticite_pdf.py` (dépôt mobiTrace). Pour toute question : équipe technique DGFIP.